TumbleStack (015-03)

Margaret Schneider
Maya Hernandez-Diaz
Summer Smith
Jackson Erb

CSCI 3308
December 5, 2025

Project Description:
The goal for this project was to create a journaling application that could be useful for people who want to get into journaling but don't really know where to begin. We wanted to make it for college-aged students like us. The main technologies we used were HTML, Handlebars, Javascript, SQL, and CSS styling. Our application itself is designed to generate three random prompts everyday for the user to choose one and answer upon logging into (or registering) their account. From there, within the application, the user can view other responses to the daily prompts, including seeing their friends responses, resequenced with priority to show your already added friends. You have the ability to like and comment on user's posts within the feed as well. You can also add new friends directly through the main feed page, or you can search for new friends and view previously added friends on the friend page. On the journal page, the user has the ability to create new journal entries (either guided or free-write) separate from their daily entry. On this page, the user can also view their archived daily, guided, and free-write entries.

Project Tracker (GitHub Project Board):
https://github.com/orgs/CU-CSCI3308-Fall2025/projects/31/views/1

Link to GitHub Repository:
https://github.com/CU-CSCI3308-Fall2025/group-project-015-03

Video Demo:
https://youtu.be/rj76MqYeUsc

Deployed App:
https://group-project-015-03.onrender.com

Contributions

Margaret:

I used Handlebars, CSS, SQL and Javascript. I created a journal page that allowed users to enter new journal entries (either guided or unguided) and view their old entries. I created the ability to search for friends, add friends, unfriend, and view current friends. For the feed, I made it so you can view all three prompts for the daily prompts, and your friends responses to them while prioritizing and highlighting your friends' responses at the top. You can also view user profiles and add new friends from the feed. I also created the ability to make a user's profile and customize their profile.

Summer:

I designed the databases and implemented them in SQL. I then made sure that the user profile information, login information, and journal entries were saved to the database. I also later added tables to support the friends functionality, both in terms of pending friend requests and tracking a user's friends. The other main thing I worked on was integrating the Spotify API. This proved to be much more challenging than I anticipated, since their API is incredibly finicky and hard to decode. It was working before the demo, did not work during the demo, but worked when I went to check it while writing this. However, I was able to connect my spotify account and save my top five tracks in the database. I spent a couple hours trying to translate this data onto the webpage, but ran out of time.

Jackson:

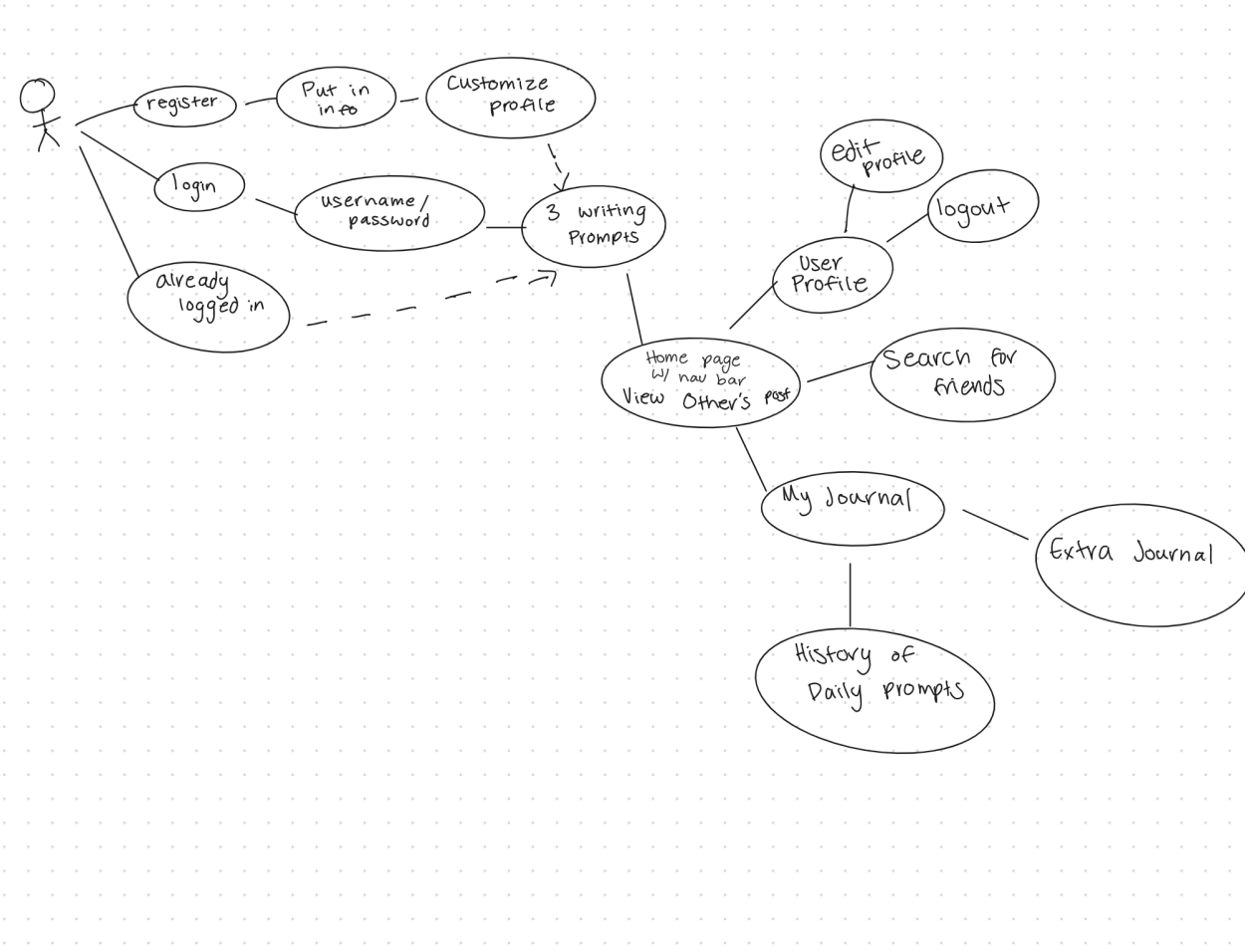
I used Handlebars, JavaScript, and SQL primarily, while also doing package management, docker maintenance, and render maintenance. The main thing I did was making sure that the frontend connected with the backend seamlessly like making sure pages that interacted with the database were persistent. I maintained multiple aspects of the project and mostly acted as the support group member for everyone to make sure that everyone was on the same page and didn't have any issues when different sides of the app implemented features. The main feature I developed was all of the initial Handlebars partials, pages, and logistics outside of the Handlebars helpers that are defined in the index.js file and some of the newer pages and partials as well.

Maya:

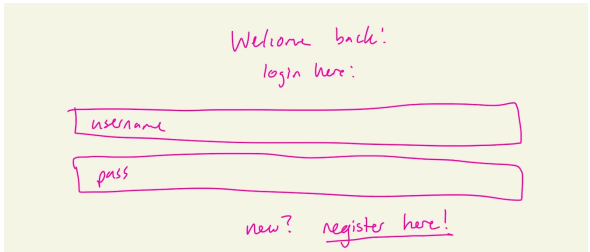
Throughout the development of Tumble Stack, I contributed to both the technical foundation of the app and the overall presentation of our work. I created the demo users database, designing the schema and populating it with realistic sample accounts so our team could test authentication, journaling, and friend interactions without manually creating new users. I also built the initial journal page skeleton, setting up the Handlebars structure, connecting it to the backend routes, and ensuring entries could be saved to and retrieved from the database. In addition to my technical work, I designed the project presentation in Canva, creating a cohesive visual theme and organizing the content so it reflected our application's style and functionality. I collaborated

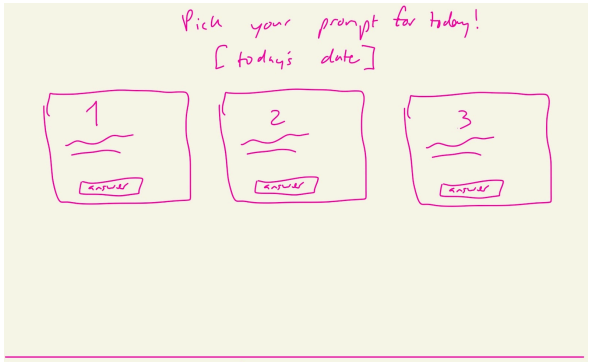
closely with my team during weekly meetings, helped troubleshoot frontend rendering and spacing issues, and assisted with GitHub branch management. Overall, my contributions supported both the app's core functionality and the clarity and professionalism of our final presentation.

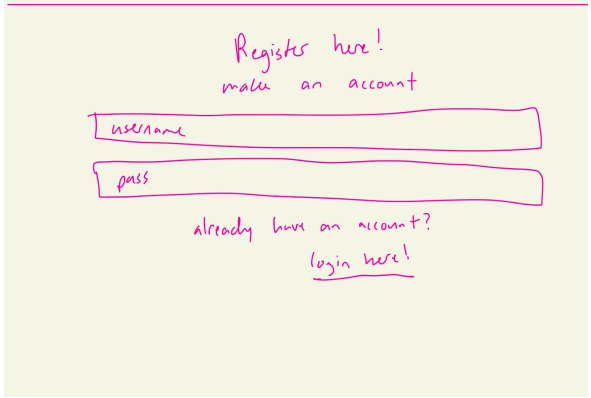
Case Diagram

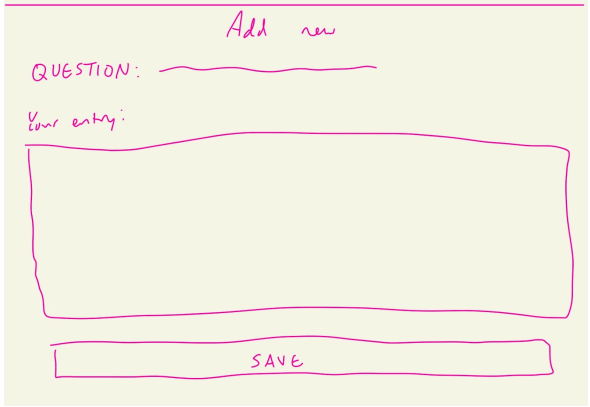


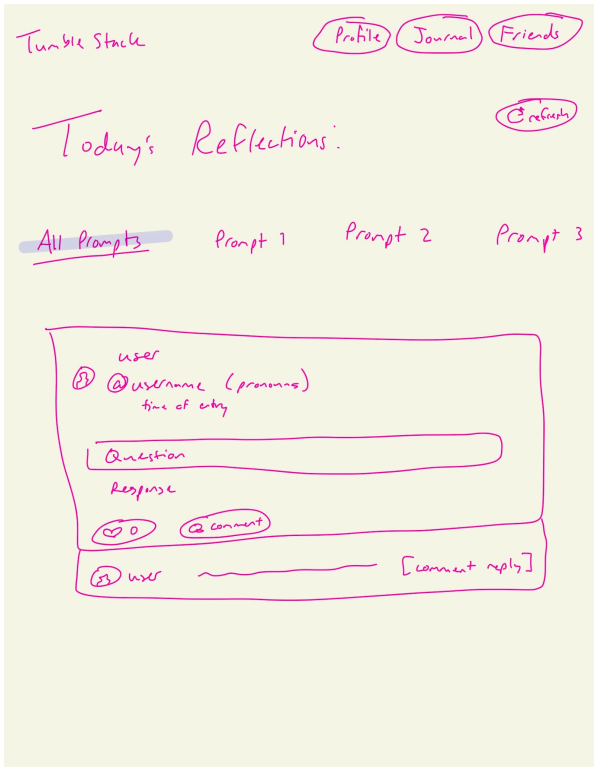
Wireframes

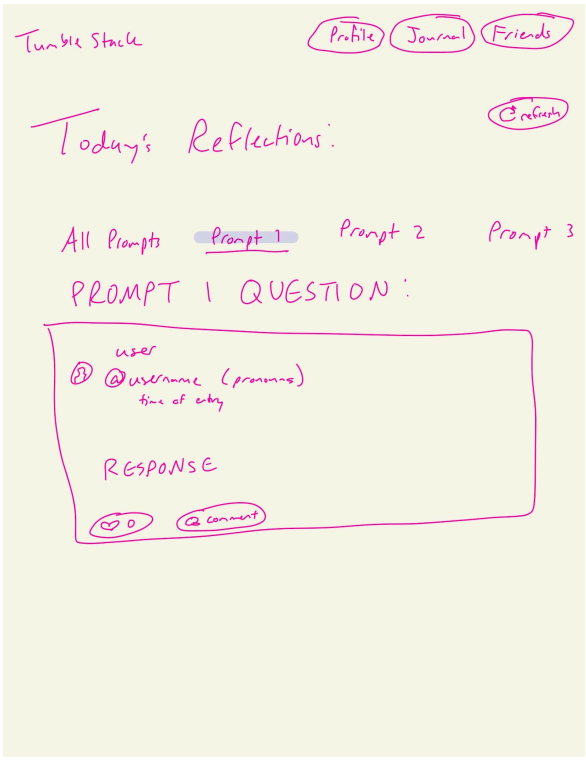

Welcome back!
login here:
username
pass
new? register here!

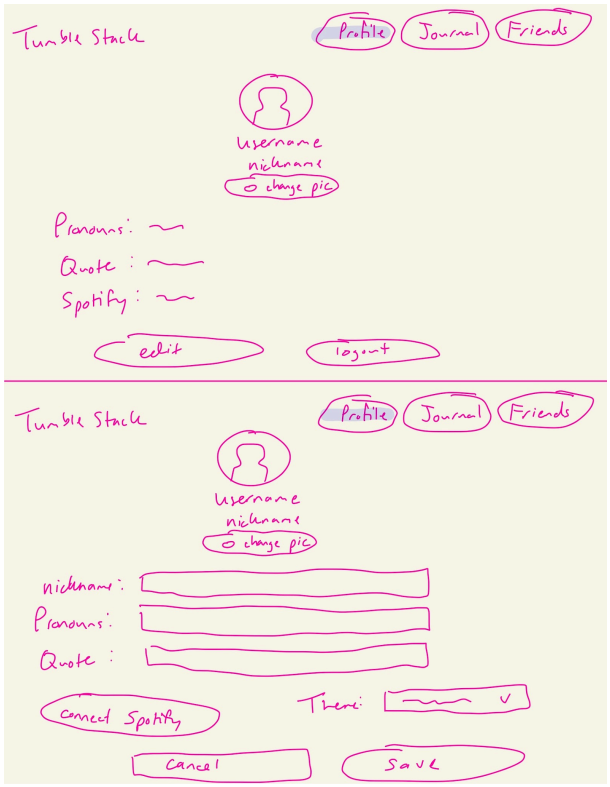

Pick your prompt for today!
[today's date]
1
answer
2
answer
3
answer

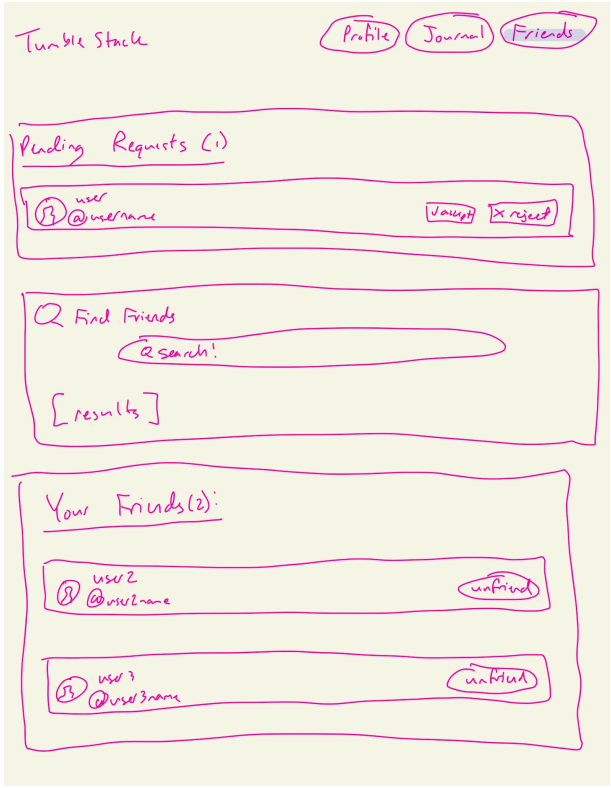

Register here!
make an account
username
pass
already have an account?
login here!

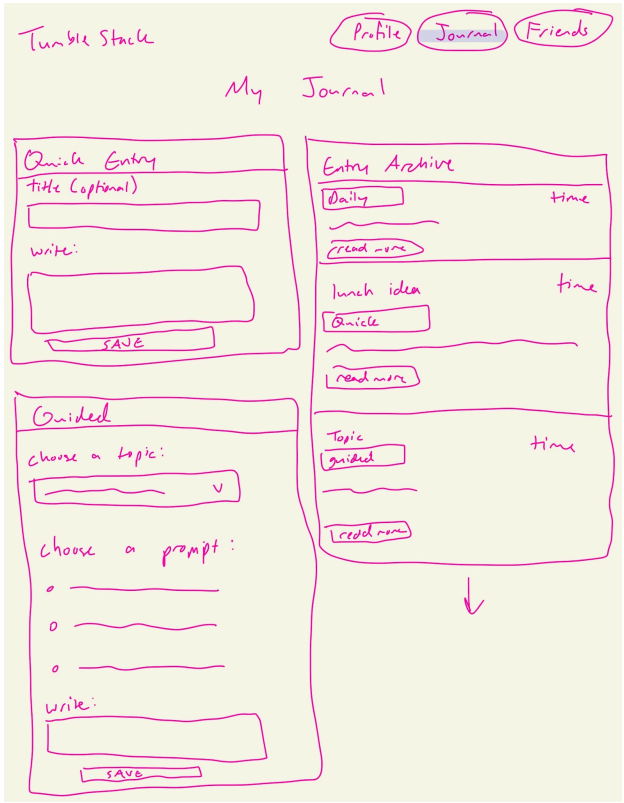

Add new
QUESTION:
Your entry:
SAVE


Tumble Stack
Profile
Journal
Friends
Today's Reflections:
refresh
All Prompts
Prompt 1
Prompt 2
Prompt 3
user
@username (pronouns)
time of entry
Question
Response
comment
user
[comment reply]


Tumble Stack
Profile
Journal
Friends
Today's Reflections:
refresh
All Prompts
Prompt 1
Prompt 2
Prompt 3
PROMPT 1 QUESTION:
user
@username (pronouns)
time of entry
RESPONSE
comment


Tumble Stack
Profile
Journal
Friends
username
nickname
change pic
Pronouns:
Quote:
Spotify:
edit
logout
Tumble Stack
Profile
Journal
Friends
username
nickname
change pic
nickname:
Pronouns:
Quote:
connect Spotify
Theme:
cancel
save


Tumble Stack
Profile
Journal
Friends
Pending Requests (1)
user
@username
accept
reject
Find Friends
search!
[results]
Your Friends (2):
user2
@user2name
unfriend
user3
@user3name
unfriend


Tumble Stack
Profile
Journal
Friends
My Journal
Quick Entry
Title (optional)
write:
SAVE
Guided
choose a topic:
choose a prompt:
write:
SAVE
Entry Archive
Daily
time
read more
lunch idea
time
Quicks
read more
Topic
guided
time
read more

Test Results

Our test cases follow the UAT plan in our github repository under MilestoneSubmissions. All of our test cases passed and we mainly tested the /journal, /register, and /login routes. We mainly tested for whether or not a user is logged in and if they can access pages that can only be accessed once logged in as well as testing to see if users can submit empty aspects of journal entries. The main issue we had when implementing these tests is that we needed to correctly respond with the correct error code and tailor our testing scripts to read the errorMessage partial. Overall, our bounds for edge cases work and correctly reject users when features aren't used correctly.

Specific test cases ran:

1. /register should register a new user successfully when the user is unique
2. /register should return a 400 error code for a duplicate username
3. /login should login successfully with valid credentials
4. /login should fail to login with an incorrect password
5. /login should fail for a non-existent username
6. /login should fail when either field is empty
7. /journal/new should successfully enter in a journal entry when there's content
8. /journal/new should successfully create a journal entry with a default title when the title field is blank
9. /journal/guided should create a guided journal entry successfully when a prompt is selected and there is content
10. /journal/new should fail when there's no content
11. /journal/guided should fail when there's no prompt selected
12. /journal/new should not be accessible to non logged-in users
13. /journal/new should reject entries that are invalid

Testing logs:

```
web-1  |   Testing /register API (Positive)
web-1  | Database connection successful
web-1  | Registered new user: test1764967850622
web-1  |     ✓ Positive: should register a new user successfully (201ms)
web-1  |
web-1  |   Testing /register API (Negative)
web-1  | Registered new user: duplicateUser
web-1  |     ✓ Negative: should return 400 for duplicate username (107ms)
web-1  |
web-1  |   Testing /login API (Positive)
web-1  | Registered new user: test1764967850930
```

```
web-1  | User logged in: test1764967850930
web-1  |     ✓ Positive: should login successfully with valid credentials (184ms)
web-1  |
web-1  |   Testing /login API (Negative)
web-1  | Registered new user: test1764967851115
web-1  |     ✓ Negative: should fail login with incorrect password (176ms)
web-1  |     ✓ Negative: should fail login for non-existent username
web-1  |     ✓ Negative: should fail login when fields are empty
web-1  |
web-1  |   Journal Entry Feature Tests
web-1  | Registered new user: user1764967851322
web-1  | User logged in: user1764967851322
web-1  | Journal entry saved for user1764967851322
web-1  |     ✓ JE-P1: Should create a new quick-entry journal successfully (268ms)
web-1  | Registered new user: user1764967851590
web-1  | User logged in: user1764967851590
web-1  | Journal entry saved for user1764967851590
web-1  |     ✓ JE-P2: Should create entry with default title when title is blank (226ms)
web-1  | Registered new user: user1764967851816
web-1  | User logged in: user1764967851816
web-1  | Guided journal entry saved for user1764967851816 (topic: grateful)
web-1  |     ✓ JE-P3: Should create a guided journal entry successfully (195ms)
web-1  | Registered new user: user1764967852012
web-1  | User logged in: user1764967852012
web-1  |     ✓ JE-N1: Should refuse journal entry with empty content (176ms)
web-1  | Registered new user: user1764967852189
web-1  | User logged in: user1764967852189
web-1  |     ✓ JE-N2: Guided entry should fail when prompt not selected (179ms)
web-1  |     ✓ JE-N3: Should block /journal/new for logged-out users
web-1  | Registered new user: user1764967852374
web-1  | User logged in: user1764967852374
web-1  |     ✓ JE-N4: Should return an error for invalid DB payload (180ms)
web-1  |
web-1  |   13 passing (2s)
```